MediCore · Pruebas de rendimiento (k6)

Escenario: mitad de los usuarios virtuales reserva citas y la otra mitad consulta agenda, pacientes, catálogo y expediente (`tests/k6/medicore.js`). Se ejecuta con `scripts/perf-tests.sh <perfil>` contra el ambiente QA.

Umbrales declarados antes de ejecutar

Perfil	Carga	p95 lectura	p95 reserva	Error inesperado
Load	30 usuarios durante 3 min	< 800 ms	< 1000 ms	< 1 %
Stress	sube hasta 150 usuarios	informativo	informativo	< 5 %
Spike	salto de 10 a 200 usuarios	< 2000 ms	< 3000 ms	< 2 %

Una reserva rechazada con 409 es el comportamiento correcto (horario ocupado) y no cuenta como error. Además, al terminar debe cumplirse: ninguna cita activa solapada por médico o paciente, y ningún mensaje en la cola de mensajes fallidos.

Resultados (ambiente QA local en Docker, equipo de 12 núcleos y 16 GB)

Perfil	Peticiones	req/s	p95 lectura	p95 reserva	Error inesperado	Reservas	409	Umbrales
load	20399	75.2	5.3 ms	8.9 ms	0 %	3260	41	cumplidos
stress	41620	196.4	6.2 ms	8.6 ms	0 %	6672	186	cumplidos
spike	43478	328.0	7.4 ms	11.5 ms	0 %	6954	188	cumplidos

Después de cada perfil: 0 solapes por médico, 0 solapes por paciente, 0 eventos pendientes y 0 mensajes en la DLQ (`reports/k6/integridad.txt`, `reports/k6/corrida.txt`).

Defectos que encontró esta prueba

- **HP-01, escenario de carga:** las primeras corridas de Stress y Spike aparecieron aprobadas casi sin reservas, porque Load había llenado las agendas. Se descartaron. Ahora cada corrida crea sus médicos y falla si no alcanza un mínimo de reservas.
- **HP-02, defecto de producto:** el publicador del outbox enviaba como máximo 20 eventos cada 2 s. La API seguía rápida, pero al terminar quedaron 14 405 notificaciones pendientes. Ahora el lote es de 200 y el publicador continúa sin esperar mientras haya lotes completos. El rezago se vació en menos de un minuto y las corridas finales terminan con 0 pendientes.

Límites de esta medición

- No se alcanzó el punto de quiebre: 200 usuarios virtuales no degradaron el sistema. Cada usuario espera entre 0,5 y 1,5 s entre iteraciones.
- k6, la API y la base de datos comparten el mismo equipo, sin latencia de red real. Los números no representan una VM de Azure `Standard_B2s`.
- El p95 de lectura se beneficia de la cache de Redis y de un expediente con pocos datos.